

Лабораторная работа №3

Вариант: № 10

Тема: Получить практические навыки разработки программ с использованием объектно-ориентированного подхода.

Постановка задачи:

Для своего варианта необходимо разработать консольное приложение, моделирующее объекты определённой предметной области. В отличие от лабораторной работы №2, данные должны быть представлены объектами классов, а не словарями.

Программа должна демонстрировать:

1. создание классов;
2. создание объектов;
3. хранение состояния объектов;
4. использование методов;
5. изменение состояния объектов;
6. инкапсуляцию;
7. наследование;
8. полиморфизм;
9. композицию объектов.

Создать базовый класс Delivery.

Производные классы:

- CourierDelivery;
- PostDelivery.

Реализовать общий метод расчёта стоимости доставки.

Для каждого типа использовать собственную формулу.

Создать класс `DeliveryService`.

Краткие теоретические сведения:

Объектно-ориентированное программирование (ООП) — подход, при котором программа строится из объектов, объединяющих данные и действия над ними. В работе реализованы ключевые принципы ООП:

Создание классов и объектов. В программе заданы классы для сущностей предметной области («доставка»), на их основе создаются объекты — экземпляры с собственными данными.

Хранение состояния. Каждый объект хранит собственный набор характеристик (отправитель, адрес, вес, статус и др.), которые определяют его текущее состояние.

Использование методов и изменение состояния. Поведение объектов описано методами: они читают и обновляют состояние — например, меняют статус доставки или рассчитывают стоимость.

Инкапсуляция. Данные объекта скрыты от прямого внешнего доступа и управляются через методы класса — это защищает целостность состояния.

Наследование. Производные классы (курьерская и почтовая доставка) получают общую структуру и базовые методы от родительского класса, добавляя свою специфику.

Полиморфизм. Единый метод расчёта стоимости реализуется по-разному в каждом типе доставки: один интерфейс — разные алгоритмы.

Композиция. Сервис доставки объединяет в себе несколько объектов (разные типы доставок), управляя ими как единым комплексом, а не наследуя их свойства.

Исходный код программы:

```
class Delivery:
    def __init__(self, sender_name: str, address: str, piece: int,
weight: float):
        self._sender_name = sender_name
        self._address = address
        self._piece = piece
        self._weight = weight
        self._status = False

    def delivery_info(self):
        str_ = (f'Отправитель {self._sender_name}\n' +
                f'Адрес {self._address}\n' +
                f'Количество {self._piece}\n' +
                f'Вес {self._weight}\n' +
                f'Статус {self._status}\n')
        return str_

    def deliver(self):
        self._status = True
        return self._status

    def cost_calculate(self):
        raise NotImplementedError()

class CourierDelivery(Delivery):
    def __init__(self, sender_name: str, address: str, piece: int,
weight: float):
        super().__init__(sender_name, address, piece, weight)
        self.piece = piece
        self.weight = weight

    piece_rate = 50
    per_kg_rate = 70

    def cost_calculate(self):
        cost = self.piece * self.piece_rate + self.weight *
self.per_kg_rate
        return cost

class PostDelivery(Delivery):
    def __init__(self, sender_name: str, address: str, piece: int,
weight: float):
        super().__init__(sender_name, address, piece, weight)
        self.piece = piece
        self.weight = weight

    piece_rate = 70
    per_kg_rate = 100

    def cost_calculate(self):
        cost = self.piece * self.piece_rate + self.weight *
```

```

self.per_kg_rate
    return cost

class DeliveryService:
    def __init__(self, sendler_name: str, address: str, piece:int,
weight:float):
        self._piece = piece
        self._weight = weight
        self.courier_delivery = CourierDelivery(sender_name, address,
piece, weight)
        self.post_delivery = PostDelivery(sender_name, address,
piece, weight)
        self.delivery = Delivery(sender_name, address, piece, weight)

    def deliver(self):
        return self.delivery.deliver()

    def delivery_info(self):
        return self.delivery.delivery_info()

def main():
    a = DeliveryService('Ivan', 'SPB', 3, 6)
    print(a.courier_delivery.cost_calculate(),
a.post_delivery.cost_calculate())
    print(a.delivery.deliver())
    print(a.delivery.delivery_info())

if __name__ == '__main__':
    main()

```